TEXT MINING & DOCUMENT INTELLIGENCE SYSTEM

Information Retrieval — Assignment 2

Project Report

Dataset: BBC News articles
Application: Streamlit-based document analytics and recommendation system
Group: Group 5

GitHub Repository
https://github.com/2025ag05284-bit/IR-Text-Mining-Document-Intelligence/tree/main

1. Project Overview

The Text Mining & Document Intelligence System is a Streamlit-based application for exploring a labelled news corpus and performing end-to-end text mining and document analysis. It brings together text preprocessing, corpus analytics, inverted indexing, manually implemented TF-IDF, document similarity, supervised classification, unsupervised clustering, and content-based document recommendations.

The application allows users to inspect the dataset, configure text preprocessing, examine term statistics, classify articles, explore clusters, and find articles similar to a selected document.

2. Project Objectives

• Ingest and validate a collection of labelled news documents.

• Clean and preprocess document text for analysis.

• Analyze corpus statistics, vocabulary, duplicates, and term distributions.

• Build an inverted index and calculate TF-IDF features without a prebuilt TF-IDF estimator.

• Classify documents with a manually implemented Naive Bayes classifier.

• Cluster documents with a manually implemented K-Means algorithm.

• Evaluate classification and clustering using appropriate metrics.

• Recommend similar documents using cosine similarity over TF-IDF vectors.

• Present the workflow and results through an interactive Streamlit application.

3. Dataset Description

The project uses labelled BBC news articles. Each record represents a document with a unique document ID, article title, article text, and category label. The five categories are business, entertainment, politics, sport, and tech.

Dataset statistic	Value
Total documents	2,225
Categories	5
Unique document IDs	2,225
Total words shown in dashboard	854,490

Category	Documents	Percentage
Sport	511	22.97%
Business	510	22.92%
Politics	417	18.74%
Tech	401	18.02%
Entertainment	386	17.35%

Average article lengths shown in the dashboard are 323.48 words for business, 325.20 for entertainment, 448.88 for politics, 324.37 for sport, and 497.43 for tech.

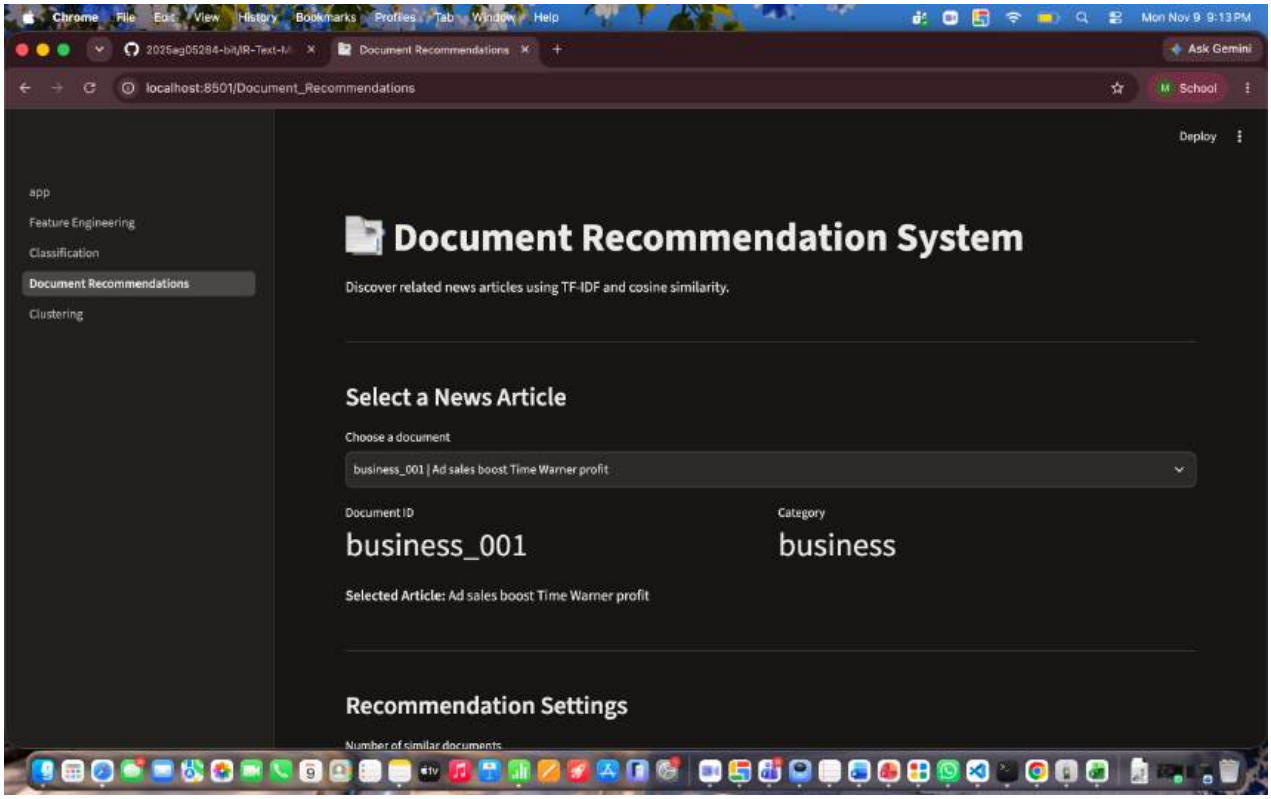


Figure 1. Dataset overview and corpus summary.

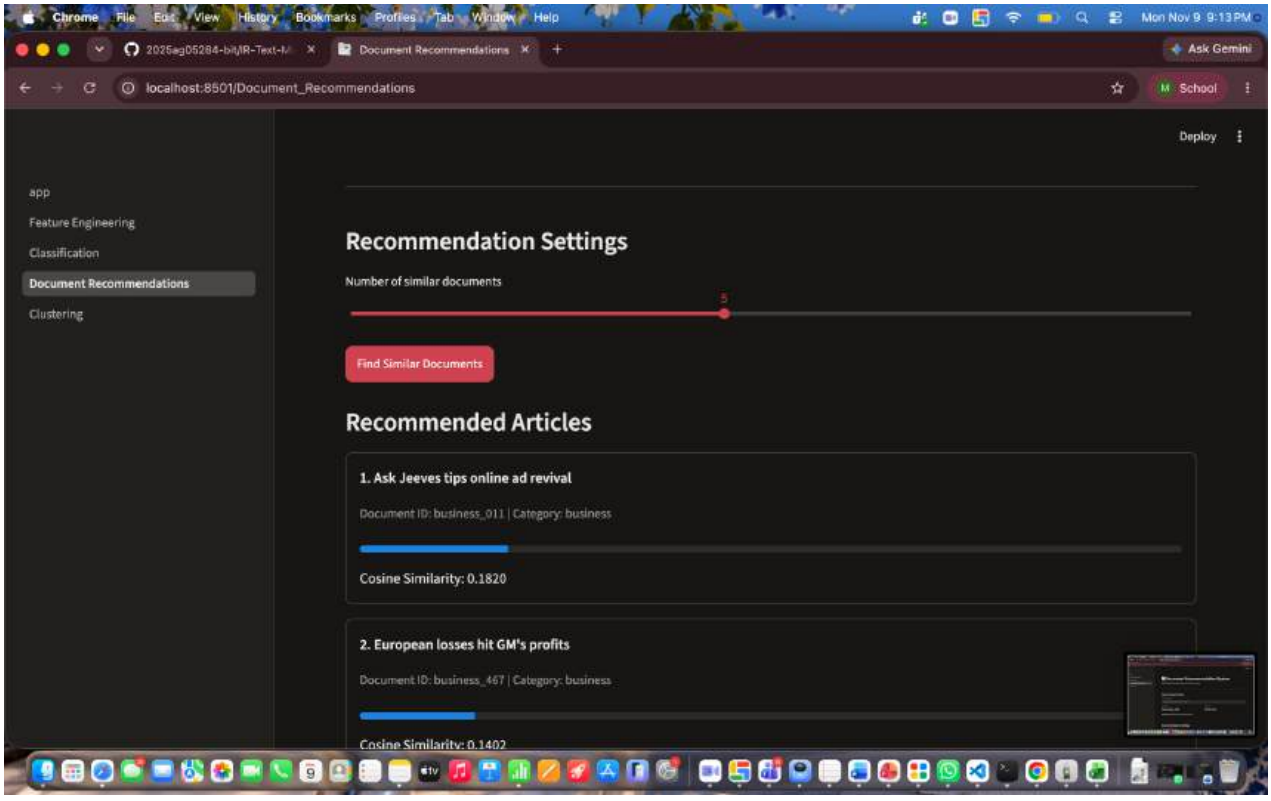


Figure 2. Category distribution by document count and percentage.

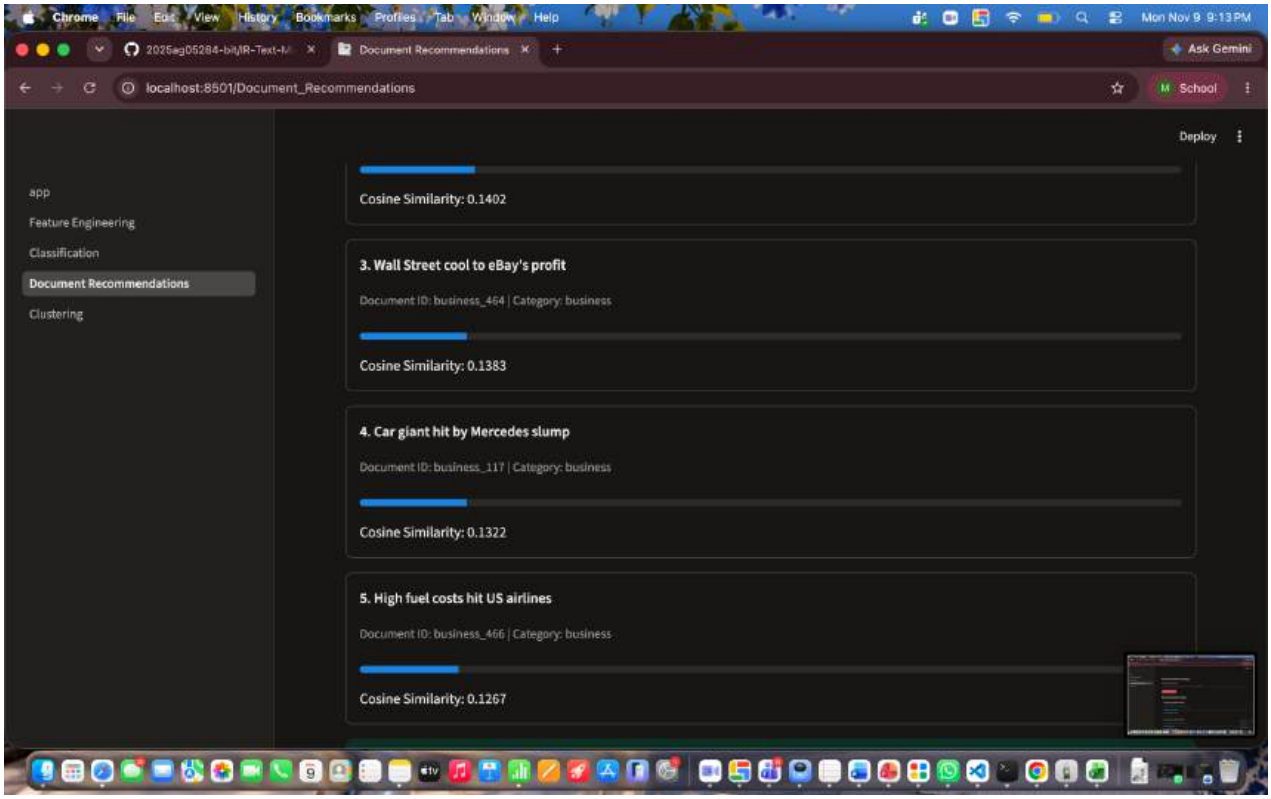


Figure 3. Category-wise statistics and document-length distribution.

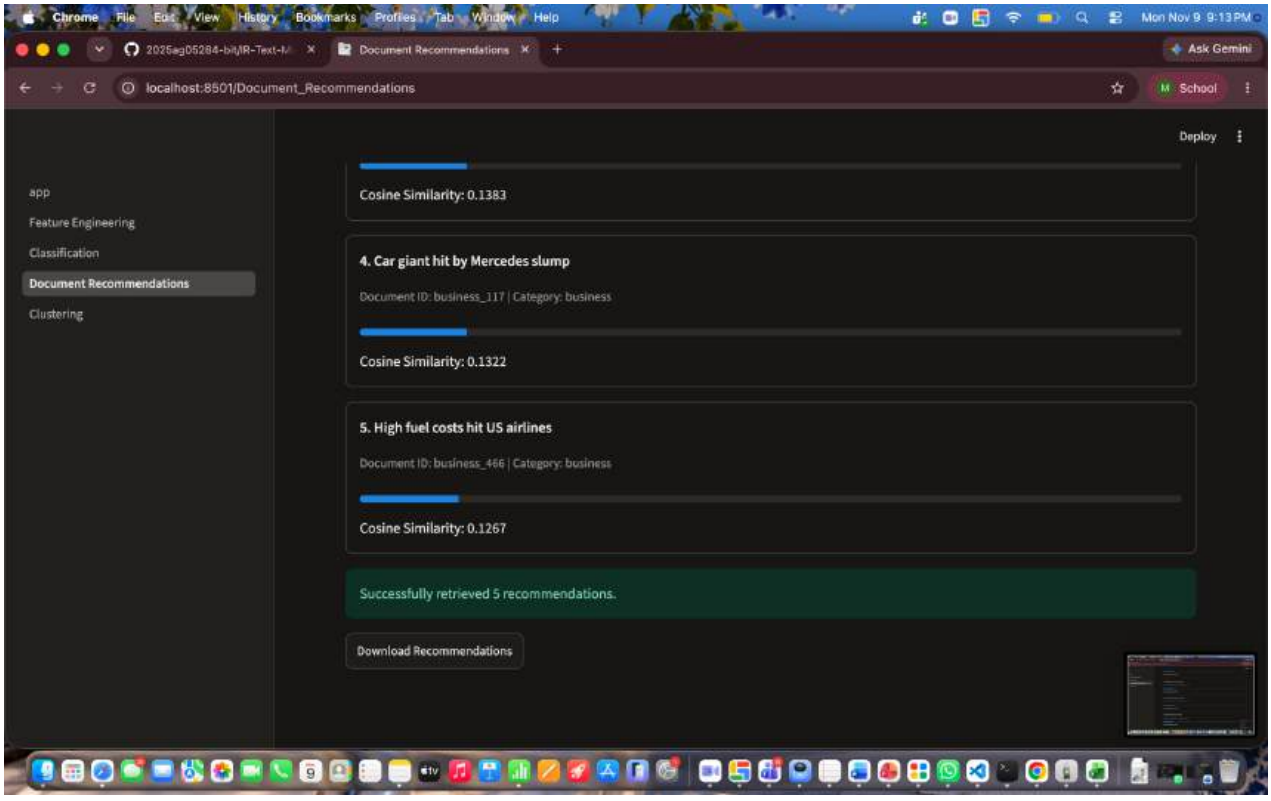


Figure 4. Document-length histogram.

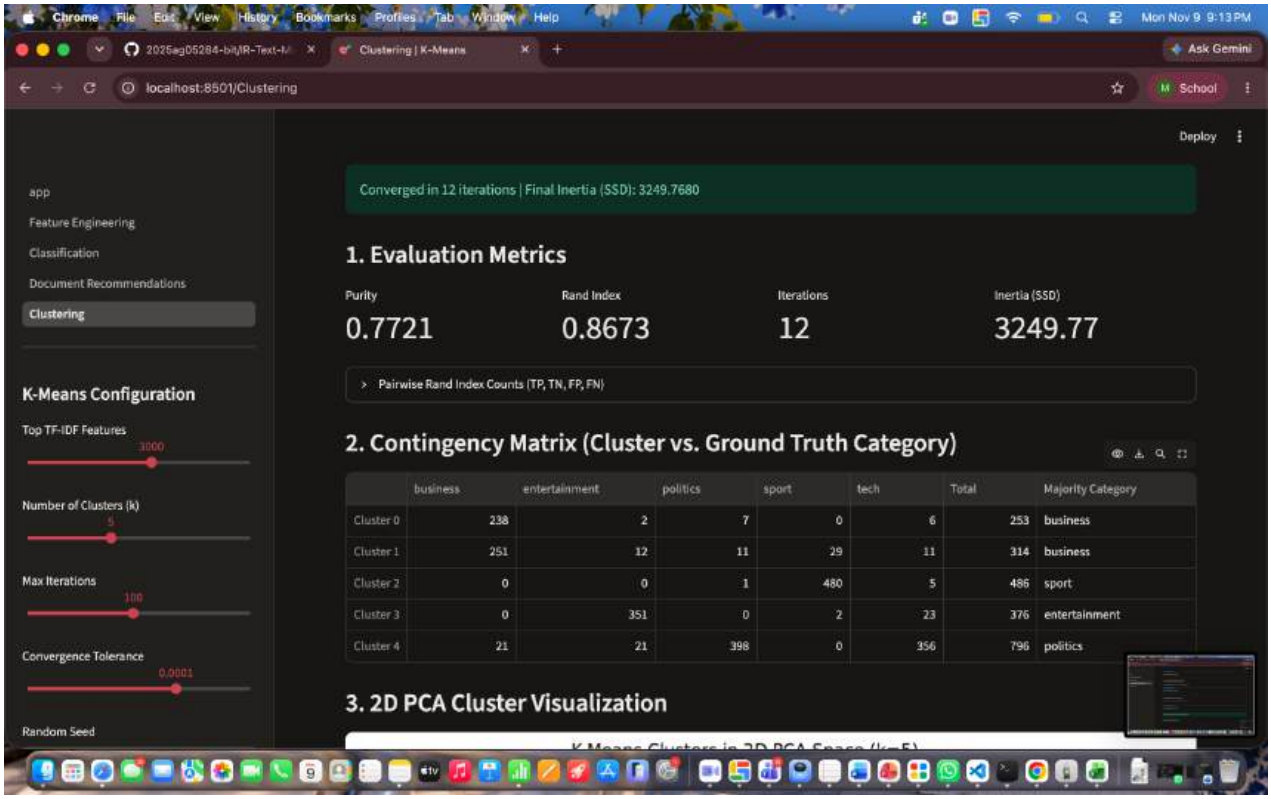


Figure 5. Vocabulary analysis and frequent terms.

4. Application Features and Workflow

The application supports dataset loading and validation, configurable text preprocessing, corpus analytics, duplicate analysis, inverted indexing, manual TF-IDF, cosine similarity, Naive Bayes classification, K-Means clustering, clustering evaluation, and Top-K document recommendations.

Stage	Operation
Dataset loading	Load news records and validate required fields and document IDs.
Preprocessing	Apply lowercase conversion, tokenization, stop-word removal, stemming and/or lemmatization, and N-gram generation.
Corpus analytics	Inspect corpus statistics, vocabulary, frequent terms, and duplicates.
Feature engineering	Build term-to-document postings and calculate TF, IDF, and TF-IDF vectors manually.
Similarity	Calculate cosine similarity between document vectors.
Classification	Train and evaluate the project-implemented Multinomial Naive Bayes classifier.
Clustering	Run project-implemented K-Means on document vectors.
Recommendation	Rank other documents by cosine similarity and display Top-K results.

5. Text Preprocessing and Feature Engineering

The preprocessing workflow includes lowercase conversion, tokenization, stop-word removal, stemming and/or lemmatization, and N-gram generation. Vocabulary analysis reports total tokens, vocabulary size, average tokens per document, and frequent-word statistics.

Configuration	Total tokens	Vocabulary size
Baseline	855,598	27,877
Stop-word removal	499,207	27,760

Configuration	Total tokens	Vocabulary size
Stemming	855,598	19,089
Lemmatization	855,598	24,852

The recorded experiments show that stop-word removal reduces the total token count. Stemming produces the smallest vocabulary, while lemmatization also reduces vocabulary size compared with the baseline. The retrieval component builds an inverted index, calculates TF and IDF manually, creates TF-IDF document vectors, and calculates cosine similarity.

6. Document Classification

The project implements a Multinomial Naive Bayes classifier. Labelled documents are divided into training and testing subsets. Word likelihoods are estimated using Laplace smoothing, and the model predicts a category for each test document. Evaluation includes accuracy, precision, recall, F1-score, and a confusion matrix.

Evaluation item	Recorded result
Accuracy	96.62%
Macro precision	96.43%
Macro recall	96.57%
Macro F1-score	96.48%
Weighted F1-score	96.63%
Test documents	444
Train/test split	80:20 stratified

7. Document Clustering and Evaluation

The project implements K-Means clustering on TF-IDF document vectors. It initializes centroids, assigns documents to clusters, updates centroids iteratively, and stops when its convergence condition is met. The application displays Purity, Rand Index, a contingency matrix, a two-dimensional PCA visualization, convergence information, and top centroid terms.

K	Purity	Rand Index
2	0.4571	0.6531
3	0.6301	0.8089
4	0.8000	0.9152
5	0.7721	0.8673
6	0.7537	0.8631
7	0.9573	0.9238
8	0.7852	0.8406

For the displayed K=5 run, the application reports Purity 0.7721, Rand Index 0.8673, convergence in 12 iterations, and final inertia (SSD) of 3249.7680. Cluster-term summaries indicate economic and financial themes in clusters 0 and 1, sports in cluster 2, entertainment in cluster 3, and politics in cluster 4. Cluster 4 contains 796 documents (35.8%) in the displayed run.

In the recorded multi-K experiment, K=7 has the highest observed Purity (0.9573) and Rand Index (0.9238).

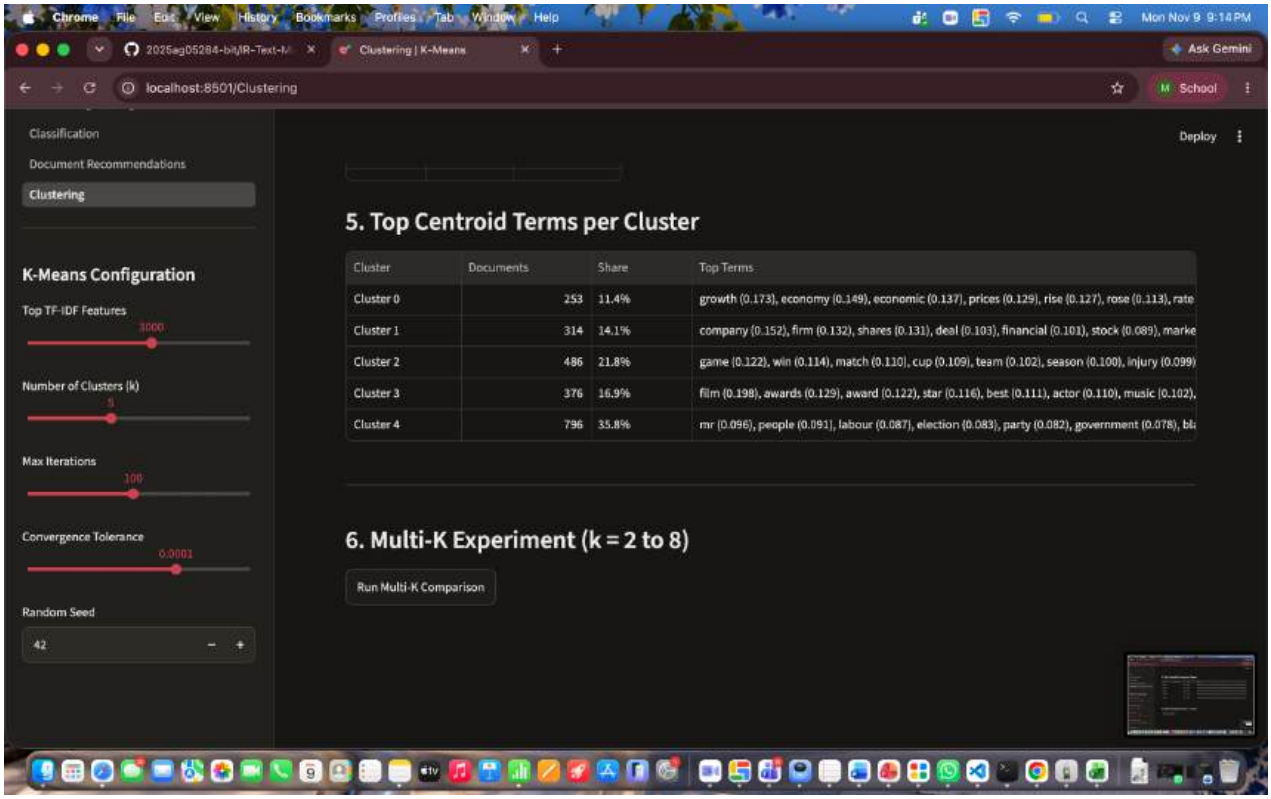


Figure 6. K-Means evaluation metrics, contingency matrix, and cluster visualization.

8. Content-Based Document Recommendations

The recommendation system lets the user select a news article and retrieve the Top-K most similar articles using cosine similarity over TF-IDF vectors. The interface displays the selected document ID and category, allows the number of recommendations to be configured, and presents ranked recommendations with document IDs, categories, and similarity scores.

The supplied example selects “Ad sales boost Time Warner profit” (document ID business_001, category business) and requests five recommendations.

Rank	Recommended article	Document ID	Category	Cosine similarity
1	Ask Jeeves tips online ad revival	business_011	business	0.1820
2	European losses hit GM's profits	business_467	business	0.1402
3	Wall Street cool to eBay's profit	business_464	business	0.1383
4	Car giant hit by Mercedes slump	business_117	business	0.1322
5	High fuel costs hit US airlines	business_466	business	0.1267

The displayed results are ordered by descending cosine similarity. The selected article is excluded. This is content-based recommendation because documents are compared using their text; collaborative recommendation would require user interaction data such as clicks, ratings, or reading histories.

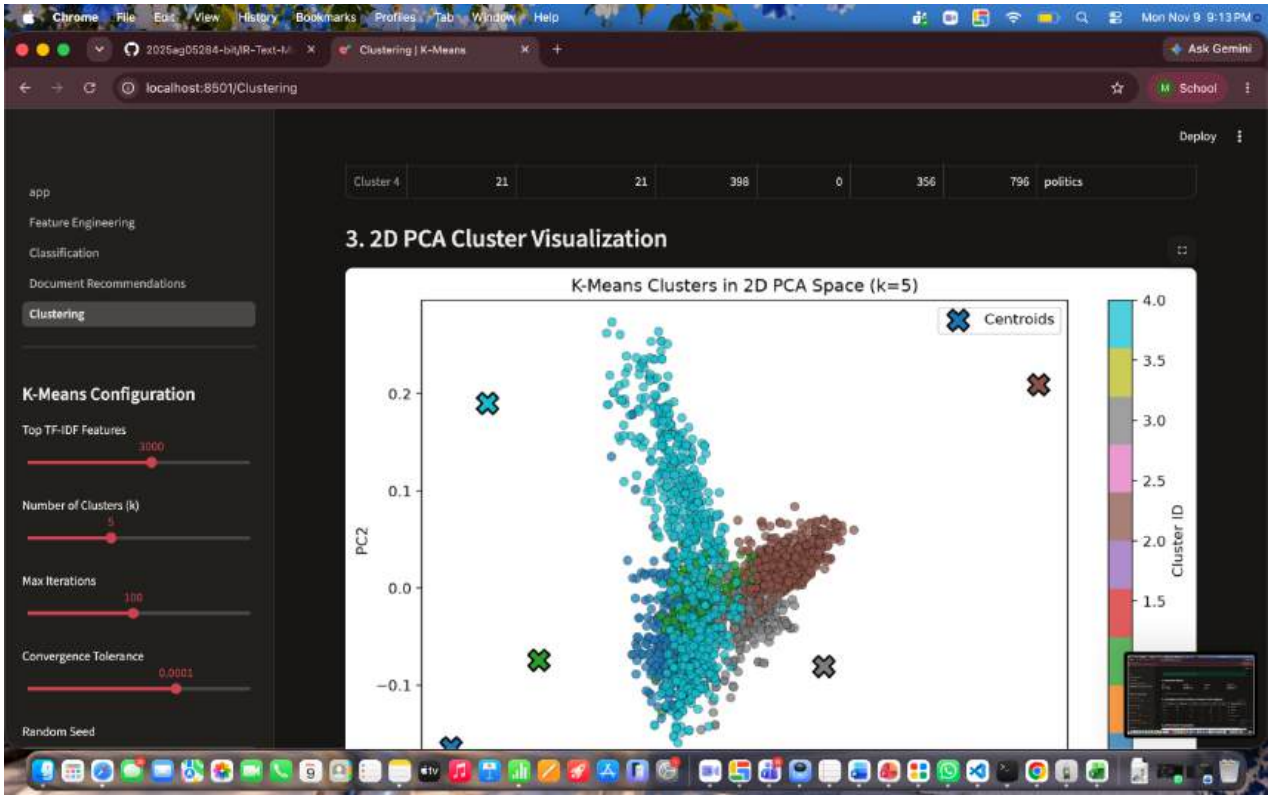


Figure 7. Recommendation page with selected article and recommendation settings.

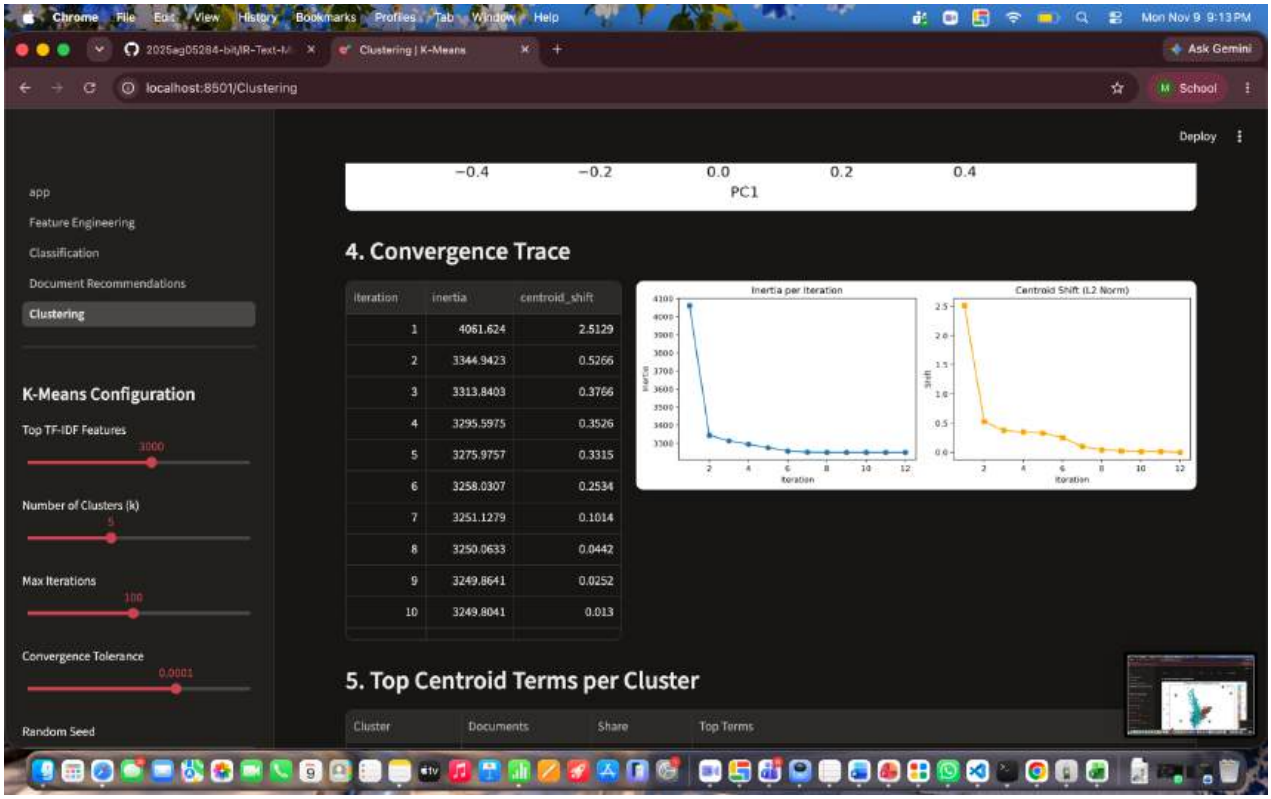


Figure 8. Ranked recommendation results with similarity scores.

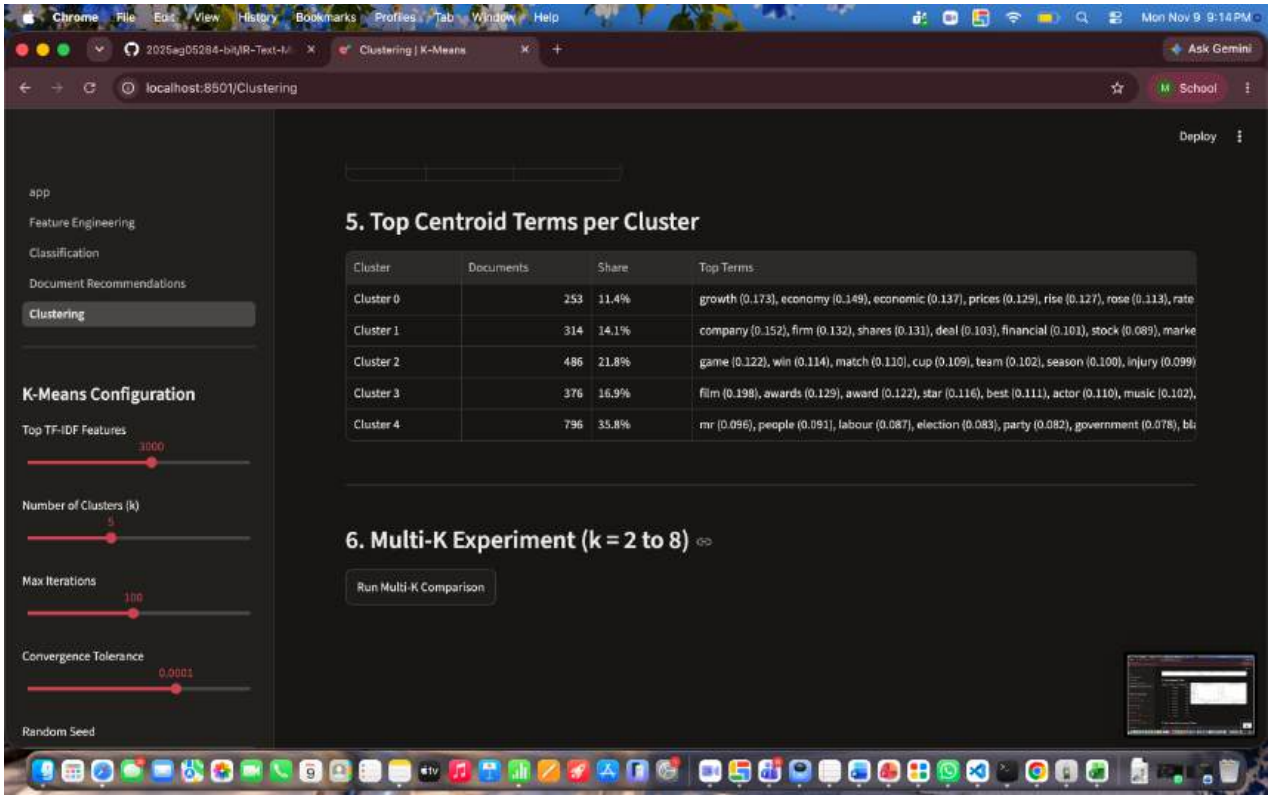


Figure 9. Remaining recommendation results and successful retrieval message.

9. Experimental Discussion

9.1 Preprocessing effects

Stop-word removal reduces total tokens from 855,598 to 499,207. Stemming reduces vocabulary size from 27,877 to 19,089, while lemmatization reduces it to 24,852. These transformations change the feature space used by retrieval and learning components.

9.2 Duplicate documents

The dataset audit identified 133 exact duplicate article-text rows among 2,225 documents. Exact duplicates can influence term statistics, similarity rankings, and clustering. They can also affect classification evaluation if copies of the same article appear in both training and testing subsets. The README describes duplicate analysis and filtering repeated/duplicate articles where duplicate metadata is available.

9.3 Business value

The system supports news analysis by organizing articles into categories, highlighting frequent terms, revealing document groupings, and retrieving related articles. Classification supports automated category assignment, clustering can expose patterns in the corpus, and recommendations reduce manual effort when searching for related coverage.

9.4 Naive Bayes and K-Means

Naive Bayes is supervised and learns from known category labels. Its recorded test performance is 96.62% accuracy and 96.48% macro F1-score. K-Means is unsupervised and forms groups from document vectors. Its displayed K=5 evaluation reports Purity 0.7721 and Rand Index 0.8673. These metrics are not directly interchangeable: classification metrics measure category prediction, while clustering metrics measure agreement between clusters and reference labels.

9.5 Limitations

Results depend on preprocessing configuration, TF-IDF representation, train/test split, random seed, and number of clusters. K-Means can produce different groupings under different initializations and values of K. Content-based recommendations depend on vocabulary overlap and article text.

10. Installation and Execution

The README lists Python 3.10 or a dependency-compatible Python version, Git, and a terminal as prerequisites. From the repository root, run:

```
git clone https://github.com/2025ag05284-bit/IR-Text-Mining-Document-Intelligence.git
cd IR-Text-Mining-Document-Intelligence
python3 -m venv .venv
source .venv/bin/activate
python -m pip install --upgrade pip
pip install -r requirements.txt
streamlit run app.py
```

Streamlit prints a local URL in the terminal, normally http://localhost:8501. Open that address in a browser. Stop the application with Ctrl+C in the terminal.

11. Team Responsibilities

Team member	Responsibility
Mubeena	Dataset ingestion and validation, preprocessing, corpus analytics, and duplicate analysis.
Aman Verma	Inverted index, manual TF-IDF, and cosine similarity.
Reshma Sri Mullapudi	Manual Multinomial Naive Bayes and classification evaluation.
Asutosh Kumar	Manual K-Means and clustering evaluation using Purity and Rand Index.
Aditya Saraswat	Content-based recommendations, Streamlit integration, documentation, and final application checks.

12. Repository Reference

Project source code, README, application modules, and project files are available at:
https://github.com/2025ag05284-bit/IR-Text-Mining-Document-Intelligence/tree/main